
Implementation Plan

Phased Fix Strategy for Property Management Module

AntiGravity Development Handoff Document

Legal Tech Application (Atrium / PracticePro)

Date: May 13, 2026

Prepared by: Z.ai Audit System

This document provides a structured, phased implementation plan for addressing the issues identified in the Comprehensive Codebase Audit Report. It is designed as a direct handoff to the AntiGravity development team, with each phase containing specific tasks, file references, acceptance criteria, and dependency mappings. The plan prioritizes security and data integrity first, then core user workflows, then architectural improvements, and finally polish. Each phase is designed to deliver a usable increment that can be verified independently before proceeding.

Table of Contents

1. Plan Overview	4
1.1 Phase Dependencies	4
2. Phase 1: Emergency Security Fixes	5
2.1 Remove Exposed Gemini API Keys	5
2.2 Fix Convex Server-Side Authentication	6
2.3 Close XSS Vectors	6
2.4 Remove MFA Debug Code from Production	7
3. Phase 2: Core Property Workflows	7
3.1 Integrate PropertyTrackingView into Active UI	8
3.2 Fix Receipt Issuance for Unit-Level Context	8
3.3 Add Unit-Level Edit Button	9
3.4 Replace Maintenance Logging Stub	10
3.5 Fix BulkEditPropertyModal Enum Mismatch	10
4. Phase 3: Financial Integration	11
4.1 Unify Payment Recording Systems	11
4.2 Build Management Fee Disbursement Workflow	12
4.3 Create Property-Finance Context Bridge	12
4.4 Move "Issue Receipt" to Quick Actions	13
5. Phase 4: Architecture Hardening	14
5.1 Memoize Context Actions and Fix God Object	14
5.2 Implement Missing Context Handlers	14
5.3 Clean Up Dead State and Duplicate Definitions	15
5.4 Incremental Type Safety Improvements	15
6. Phase 5: Polish and Discovery	16
6.1 Hydrate Missing Convex Tables	16

6.2 Receipt Template Standardization	17
6.3 End-to-End User Testing and Gap Discovery	17
7. Testing and Verification Strategy	18
7.1 Per-Phase Verification Checklist	18
8. AntiGravity Handoff Notes	19
8.1 Architecture Constraints	19
8.2 Key Files Reference	19
8.3 Communication Protocol	20
9. Risk Register	21
10. What Comes After: Next Iteration Planning	21

1. Plan Overview

This implementation plan is organized into five phases, ordered by urgency and dependency. Each phase delivers a self-contained improvement that can be tested and validated before the next phase begins. The plan is designed so that AntiGravity can work through it iteratively, shipping each phase to production (or staging) and verifying that nothing has regressed before moving forward. The total estimated effort is 5-7 weeks for a single developer, or 3-4 weeks with two developers working in parallel where dependencies allow.

The phases are structured around a core principle: fix what blocks users first, fix what risks data second, fix what slows the app third, and then improve what delights users last. This means security vulnerabilities and broken core workflows are addressed before performance optimizations and before new features. The property management module is the primary focus, as it is the newest and most actively evolving section of the application.

Phase	Focus	Timeline	Key Deliverables
Phase 1	Emergency Security Fixes	Week 1	Remove exposed API keys; fix Convex auth; close XSS vectors
Phase 2	Core Property Workflows	Weeks 2-3	Wire up tracking; fix receipt flow; add unit edit; unify payments
Phase 3	Financial Integration	Week 4	Property-Finance bridge; management fee disbursement; auto-calculations
Phase 4	Architecture Hardening	Week 5	Memoize context; fix dead state; implement missing handlers; type safety
Phase 5	Polish and Discovery	Weeks 6-7	Quick Actions expansion; UX improvements; further feature discovery

Table 1: Implementation Phases Overview

1.1 Phase Dependencies

Phase 1 (Security) has no dependencies and should begin immediately. Phase 2 (Core Workflows) can begin in parallel with Phase 1 but the receipt fixes depend on Phase 1 being complete for testing. Phase 3 (Financial Integration) depends on Phase 2 being complete because it extends the unified payment system. Phase 4 (Architecture) can begin as soon as Phase 2 is stable. Phase 5 (Polish) depends on all prior phases being complete and stable. Within each phase, tasks are listed in dependency order where applicable.

Phase	Depends On	Can Parallel With	Blocks
-------	------------	-------------------	--------

Phase 1	None	Phase 2 (partial)	Phase 2 receipt testing
Phase 2	Phase 1 (for testing)	Phase 1	Phase 3
Phase 3	Phase 2	Phase 4 (partial)	Phase 5
Phase 4	Phase 2 stable	Phase 3	Phase 5
Phase 5	All prior phases	None	None

Table 2: Phase Dependency Map

2. Phase 1: Emergency Security Fixes

PHASE 1 | EMERGENCY SECURITY | WEEK 1

This phase addresses the most critical security vulnerabilities in the application. These issues represent immediate risk: exposed API keys allow unauthorized usage at the project owner's expense, and absent server-side authentication means any client with a firmId can access or destroy any firm's data. These fixes must be deployed before any further feature work or public-facing testing.

2.1 Remove Exposed Gemini API Keys

Priority: Critical | Effort: 2-3 hours | Risk: Low

Two live Google Gemini API keys are embedded directly in the source code and are fully visible in the browser bundle. These keys can be scraped automatically from the deployed site, allowing unauthorized API usage at the project owner's expense. Additionally, API keys are passed in URL query strings which are logged by proxies, CDNs, and browser history.

Required Changes:

- Remove the hardcoded API keys from [src/utils/aiUtils.ts](#) (line 163), [src/services/geminiService.ts](#) (lines 246, 379, 460, 464, 544), and [src/components/aloe/AloeChat.tsx](#)
- Create a new Convex action ([convex/ai.ts](#) or extend existing) that proxies all Gemini API calls server-side. The action should read the API key from Convex environment variables (not the client bundle) and forward requests to the Gemini API.
- Update all client-side AI calls to use the Convex action instead of calling Gemini directly. This includes the embedding generation, chat completions, and document structuring flows.
- Move API key to Convex environment variables via `npm run convex env set GEMINI_API_KEY your_key_here`. The `VITE_GEMINI_API_KEY` environment variable should be removed entirely.
- Rotate both compromised keys immediately after deploying the fix. Old keys must be revoked in the Google Cloud Console.

Acceptance Criteria:

- No API key strings appear anywhere in the browser bundle (verify with a production build search)
- All AI features (Aloa chat, document structuring, embedding generation) continue to work through the server-side proxy
- The Gemini API key is only accessible via Convex environment variables, never exposed to the client

2.2 Fix Convex Server-Side Authentication

Priority: Critical | Effort: 6-8 hours | Risk: Medium (requires thorough testing)

The `withFirmAuth` middleware in `convex/lib/withAuth.ts` does not actually verify the caller's identity. It never calls `ctx.auth.getUserIdentity()`, proceeds even when no user is found, and uses an inefficient full-table scan. This means that 50+ Convex functions are effectively unauthenticated. Any client that knows a `firmId` can read, create, update, or delete any firm's data.

Required Changes:

- Rewrite `convex/lib/withAuth.ts` to call `ctx.auth.getUserIdentity()` as the first step. If the identity is null, throw a Convex error immediately rather than proceeding in read-only mode.
- Replace the full-table scan (`take(500)`) with a proper indexed query using the user's `tokenIdIdentifier` from `getUserIdentity()`. Create an index on the users table by `tokenIdIdentifier` if one does not exist.
- Add the `withFirmAuth` wrapper to all currently unprotected Convex functions. Start with the highest-risk ones: `getFirmData`, `purgeFirmData`, `adminDeleteUser`, `adminForceVerify`, `dumpAll`.
- For admin functions (`adminDeleteUser`, `adminForceVerify`, `getAllFirmsForAdmin`, `getAllLicenses`), add a role check that verifies the caller has an admin role before proceeding.
- Add `checkResourceOwnership` to all item-level mutations (`createItem`, `updateItem`, `deleteItem`) to ensure the item belongs to the caller's firm.
- Remove or gate debug functions: `checkEnv` (exposes env variables), `testInsertUndefined`, `dumpAll`. These should not exist in production.

Acceptance Criteria:

- All Convex queries and mutations verify the caller's identity via `getUserIdentity()` before executing
- Unauthenticated requests are rejected with a clear error message, not silently allowed
- Admin-only functions require an admin role check
- Debug functions (`checkEnv`, `dumpAll`) are removed or gated behind admin-only access
- All existing functionality continues to work with authenticated requests (run full regression)

2.3 Close XSS Vectors

Priority: High | Effort: 1-2 hours | Risk: Low

Four components use `dangerouslySetInnerHTML` with `parseAloaMarkdown()` output without passing through the existing `sanitize()` function. If any of this content comes from AI output or other users, it represents an XSS attack vector.

Required Changes:

- [src/components/aloa/NoteDetails.tsx](#) (line 34): Wrap `parseAloaMarkdown(note.content)` with `sanitize()`
- [src/components/aloa/MiniAloa.tsx](#) (line 380): Wrap `parseAloaMarkdown(msg.content)` with `sanitize()`
- [src/components/aloa/AloaChat.tsx](#) (line 1027): Wrap `parseAloaMarkdown(msg.content)` with `sanitize()`
- [src/components/MessagingView.tsx](#) (lines 270, 859): Wrap `parseAloaMarkdown(content)` and `parseAloaMarkdown(item.adminReply)` with `sanitize()`

Acceptance Criteria:

- All `dangerouslySetInnerHTML` usages pass through `sanitize()` before rendering
- Aloa chat messages, notes, and messaging content render correctly with sanitization (no visual regression)

2.4 Remove MFA Debug Code from Production

Priority: High | Effort: 15 minutes | Risk: None

Login.tsx line 110 logs the actual MFA verification code to the browser console, even in production builds. This allows anyone with access to browser dev tools to bypass MFA entirely.

Required Changes:

- [src/components/auth/Login.tsx](#) (line 110): Wrap the `console.log` in a DEV mode check: `if (import.meta.env.DEV) console.log(...)`
- Additionally, audit all `console.log` statements across the codebase and gate non-error logs behind `import.meta.env.DEV`

Acceptance Criteria:

- MFA codes do not appear in the browser console in production builds
- Debug `console.log` statements are suppressed in production; error-level logs remain

3. Phase 2: Core Property Workflows

PHASE 2 | CORE PROPERTY WORKFLOWS | WEEKS 2-3

This phase fixes the broken core workflows that users encounter daily. The `PropertyTrackingView` component, which contains the full unit-level tracking experience, is imported but never rendered. The

receipt issuance flow discards unit-specific context. Maintenance logging is a stub. These issues collectively make the property management module confusing rather than efficient, which is the opposite of the intended user experience.

3.1 Integrate PropertyTrackingView into Active UI

Priority: Critical | Effort: 4-6 hours | Risk: Medium (UI integration)

PropertyTrackingView contains a fully built timeline, rent history table, maintenance Kanban board, lease setup form, manual event addition, and receipt download. It is imported in PropertyDetailView but never rendered. The "Add Payment" and "Save Record" buttons that the user reported as not working are part of this orphaned component. Integrating it into the active UI is the single highest-impact fix for the property module.

Required Changes:

- In `src/components/details/PropertyDetailView.tsx`, replace the current inline unit tracking stubs in the Units tab with the PropertyTrackingView component. Each unit card should either expand to show PropertyTrackingView inline, or clicking a unit should navigate to a dedicated tracking view.
- Wire the `onUpdate` callback of PropertyTrackingView to the data persistence layer. The callback should call `updateItem("properties", updated)` to persist changes to Convex. Currently, `onUpdate` is a no-op because the parent does not pass it.
- Ensure the rent payment recording within PropertyTrackingView also writes to the Convex `ledger_entries` table (in addition to `property.rentPaymentHistory`) to maintain consistency with the `RecordRentPaymentModalWrapper` flow. This is a stepping stone to the unified payment system in Phase 3.
- Add the PropertyTrackingView to the Summary tab as well, so that single-tenant properties can see their tracking timeline without navigating to a unit.

Acceptance Criteria:

- Clicking "Add Payment" inside a unit shows the payment form, and clicking "Save Record" persists the payment to both `property.rentPaymentHistory` and Convex `ledger_entries`
- The Activity Timeline shows events and auto-populates when rent is collected or maintenance is logged
- The Maintenance Kanban board displays maintenance records and allows status transitions (reported, in progress, escalated, fulfilled)
- Lease setup form allows editing lease terms and they persist after page refresh

3.2 Fix Receipt Issuance for Unit-Level Context

Priority: Critical | Effort: 3-4 hours | Risk: Low

When a user clicks "Issue Receipt" from a unit cog menu, the ModalManager discards the `modalContext` containing unit-specific data (`unitName`, `tenantName`, `rentAmount`) and passes only the parent property

to CollectRentModal. This results in receipts with incorrect tenant names and rent amounts for multi-unit properties. Additionally, the ReceiptDetailView shows the landlord name instead of the tenant name in the "Received From" field.

Required Changes:

- In [src/components/modals/ModalManager.tsx](#) (lines 323-330), pass modalContext as a prop to CollectRentModal. Follow the pattern already used by RecordRentPaymentModalWrapper (lines 138-148) which correctly receives and uses modalContext.
- Modify [src/components/modals/CollectRentModal.tsx](#) to accept an optional modalContext prop. When modalContext is provided (unit-level receipt), use modalContext.tenantName instead of the property-level tenant, and modalContext.rentAmount instead of property.rentalDetails.rentAmount. Pre-fill the rent amount field from this value rather than requiring the user to enter it.
- In [src/components/details/ReceiptDetailView.tsx](#) (lines 40-46), fix the "Received From" field. For property rent receipts, the payer is the tenant, not the client/landlord. Add logic to detect when a receipt is for a property payment and show the tenant name accordingly.
- Remove the valuations dialogue for properties that are not listed for sale. In [PropertyForm.tsx](#) (line 560) and [PropertyDetailView.tsx](#) (line 504), gate the valuation field behind a "category === For Sale" condition. For rental and management properties, the valuation field is irrelevant and creates confusion.

Acceptance Criteria:

- Issuing a receipt from a unit cog menu shows the correct unit tenant name and rent amount
- The rent amount field is pre-populated from the unit's agreed rent amount
- Receipts show the tenant name as "Received From", not the landlord name
- Valuation field is hidden for non-sale properties

3.3 Add Unit-Level Edit Button

Priority: High | Effort: 2-3 hours | Risk: Low

Currently, the only way to edit individual units is through the ExpandablePropertyGroup sidebar component. The Units tab in PropertyDetailView has no edit button on unit cards. The main Edit button in the header opens the full property form, which is confusing when the user wants to edit a specific unit. The user requested a per-unit edit button placed below the main Edit button and beside the units list, with scrollable support for many units.

Required Changes:

- In [src/components/details/PropertyDetailView.tsx](#), add an "Edit Unit" button to each unit card in the Units tab. This should be placed below the main property Edit button in the header, or as a button beside each unit card.
- When clicked, open a PropertyForm modal pre-populated with the unit's data. Use the existing `openModal("editProperty", unit.id, { contactId })` flow, but ensure the PropertyForm detects it is

editing a unit (not a standalone property) and adjusts the form fields accordingly.

- Ensure the units list container has `overflow-y: auto` with a `max-height` constraint so that it scrolls when there are many units. Verify that the edit button and other UI elements do not overlap with the scrollable area.
- Test with 10+ units to verify scrolling behavior and button placement.

Acceptance Criteria:

- Each unit card has an "Edit" button that opens a pre-populated edit form for that specific unit
- The main "Edit" button in the header continues to edit the parent property
- The units list scrolls properly when there are many units
- No UI elements overlap, regardless of the number of units

3.4 Replace Maintenance Logging Stub

Priority: High | Effort: 3-4 hours | Risk: Low

The "Log Maintenance" option in the unit cog menu only toggles the unit status to "Maintenance". It does not create a `MaintenanceRecord` with issue details, priority, vendor info, or estimated cost. The `PropertyTrackingView` already has a full maintenance form (`addType === "maintenance"` in `handleSave`), but since that component is orphaned, this form is inaccessible.

Required Changes:

- Replace the inline stub in `PropertyDetailView.tsx` (lines 845, 879) with a proper maintenance logging modal or inline form. Extract the maintenance form logic from `PropertyTrackingView`'s `handleSave` into a reusable `MaintenanceForm` component.
- The form should capture: issue description, priority (low/medium/high/critical), category (plumbing/electrical/structural/other), estimated cost, vendor/contractor (optional), and expected completion date.
- On submission, create a `MaintenanceRecord` in the `property.maintenanceHistory` array AND log a timeline event. Persist via `updateItem` to Convex.
- The maintenance record should appear in the `PropertyTrackingView` maintenance Kanban once that component is integrated (Task 3.1).

Acceptance Criteria:

- Clicking "Log Maintenance" opens a form, not just a status toggle
- Maintenance records persist to the database and survive page refresh
- Maintenance records appear in the Kanban board (once integrated)

3.5 Fix BulkEditPropertyModal Enum Mismatch

Priority: Medium | Effort: 1 hour | Risk: Low

Required Changes:

- In [src/components/modals/BulkEditPropertyModal.tsx](#), align the status dropdown options with the `PropertyStatus` enum used throughout the app (Occupied, Vacant, Listed). Remove the incorrect values (Available, Leased, For Sale, Maintenance, Under Renovation).
- Similarly align the category dropdown with the app's category enum (Tenanted Property, Property For Sale, etc.) instead of the generic (Residential, Commercial).

Acceptance Criteria:

- Bulk edit sets status and category values that are recognized by the app's filtering and display logic

4. Phase 3: Financial Integration

PHASE 3 | FINANCIAL INTEGRATION | WEEK 4

This phase addresses the fundamental disconnect between the property management module and the finance context. Currently, property earnings (rent) and expenses (maintenance) are not integrated with the `FinanceContext`, meaning the finance page does not reflect property financial activity. Additionally, the dual payment recording systems (`ledger_entries` vs `rentPaymentHistory`) must be unified, and the management fee disbursement workflow needs to be built out to support the user's stated business model where the property owner collects rent directly and disburses the management fee.

4.1 Unify Payment Recording Systems

Priority: Critical | Effort: 5-6 hours | Risk: High (data migration)

Warning: This task involves data model changes and requires a migration strategy. Ensure a backup of the Convex database is taken before proceeding.

Required Changes:

- Design a unified payment record model that satisfies both the property tracking needs (rent history, receipt generation) and the financial reporting needs (ledger entries, service charges). The Convex `ledger_entries` table should be the single source of truth for all financial transactions.
- Modify [CollectRentModal.tsx](#) to write to `ledger_entries` (via `api.sentry.addLedgerEntry`) in addition to `property.rentPaymentHistory`. The `rentPaymentHistory` array can be derived from `ledger_entries` at read time, or kept as a denormalized cache that is always written in sync with the ledger.
- Modify [PropertyTrackingView](#) (once integrated) to write to `ledger_entries` when recording rent payments, rather than only updating the local property object.
- Add a "sync" utility that backfills any existing `rentPaymentHistory` entries into `ledger_entries` for properties that were created before this fix. This should run as a Convex migration.
- Update the property financial reports ([PropertyReports.tsx](#)) to pull data from `ledger_entries` rather than computing from `rentPaymentHistory` arrays.

Acceptance Criteria:

- Recording a rent payment (via any path) creates a single consistent record visible in both the property rent history and the financial ledger
- The property's total income in RevenueEngine matches the sum of individual unit receipts
- No duplicate entries appear in either the rent history or the ledger
- Historical data is preserved through the migration utility

4.2 Build Management Fee Disbursement Workflow

Priority: High | Effort: 4-5 hours | Risk: Medium

The user has stated that as a property manager, they do not collect rent to themselves directly. The property owner receives the rent, and then disburses the management fee to the property manager. This is not the only possible model (some managers collect and remit), so the system must support both. The current CollectRentModal only generates a fee invoice; it does not track the disbursement lifecycle.

Required Changes:

- Add a "collection model" field to the property schema and form: "Manager Collects" (manager receives rent, remits net to owner) or "Owner Collects" (owner receives rent, disburses fee to manager). Default to "Owner Collects" based on the user's stated preference.
- When collection model is "Owner Collects", the receipt flow should: (a) issue receipt to tenant for full rent amount, (b) create a payable entry for the management fee (visible to the owner), and (c) track the disbursement status (pending, disbursed, overdue).
- Add a "Disbursements" tab or section in the property financial view that shows pending management fee disbursements. The property owner should be able to mark fees as disbursed, which updates the financial records and creates a receipt for the manager.
- Link the disbursement workflow to the Finance page. Property management fee income should appear as earnings in the finance context (tagged as "Property Management Fee"), and property expenses should appear as expenses (tagged as "Property Maintenance").

Acceptance Criteria:

- Property form includes a "collection model" field (Manager Collects / Owner Collects)
- In "Owner Collects" mode, receipts are issued to tenants for the full rent amount, and management fees appear as payables
- Disbursements can be tracked, marked as paid, and generate receipts for the manager
- Property earnings and expenses are visible in the Finance page with appropriate tagging

4.3 Create Property-Finance Context Bridge

Priority: High | Effort: 4-5 hours | Risk: Medium

The FinanceContext currently provides invoices, timeEntries, and expenses, but these are legal billing constructs with no property financial awareness. Property earnings (rent) and expenses (maintenance) are not aggregated into any finance context. The RevenueEngine must manually traverse coreState.properties to compute financials. This task creates a proper integration layer.

Required Changes:

- Create a [src/contexts/PropertyFinanceContext.tsx](#) that aggregates property financial data from ledger entries, service charges, and maintenance costs. This context should provide computed values such as: totalPropertyIncome, totalPropertyExpenses, netPropertyIncome, occupancyRate, arrearsTotal, and managementFeesOwed.
- Integrate this context into the existing provider hierarchy. It should sit between FinanceProvider and DocumentProvider, reading from CoreContext for raw property data and from FinanceContext for existing financial records.
- Add a visual indicator on the Finance page that distinguishes property-sourced earnings from legal-service earnings. This could be a badge, a different color, or a separate section. Allow users to filter or aggregate these categories as needed.
- Wire the PropertyFinanceContext into the RevenueEngine and PropertyReports components, replacing their current manual computation logic with the context's computed values.

Acceptance Criteria:

- Finance page shows property earnings separately from legal service earnings, with a clear visual distinction
- Users can filter the Finance view to show only property income, only legal income, or both
- RevenueEngine and PropertyReports use the PropertyFinanceContext for computed values

4.4 Move "Issue Receipt" to Quick Actions

Priority: Medium | Effort: 2-3 hours | Risk: Low

The user has requested that issuing a receipt should fall under Quick Actions and should actually be quick: at this stage, the user should be able to confirm or edit the contents of the receipt and have it appear with a consistent style and standard for all tenants. Currently, Quick Actions only appear for non-leased, single-unit properties. Multi-unit properties are excluded from the Quick Actions section entirely.

Required Changes:

- In [PropertyDetailView.tsx](#) (lines 585-635), extend the Quick Actions section to appear for all property types, including multi-unit properties. For multi-unit properties, Quick Actions should include a per-unit "Issue Receipt" option.
- The "Issue Receipt" Quick Action should open a streamlined receipt confirmation view (not the full CollectRentModal). This view should pre-populate with the tenant name, rent amount, and payment period, and allow the user to confirm or edit before generating the receipt.

- Receipts generated through Quick Actions should use the same PDF template and styling as receipts generated through other paths, ensuring a consistent standard for all tenants.
- After issuing a receipt through Quick Actions, provide immediate visual confirmation (a toast and a preview of the receipt) rather than just closing the modal.

Acceptance Criteria:

- "Issue Receipt" appears as a Quick Action for all property types
- The receipt flow is quick: pre-populated, confirm or edit, generate, done
- All receipts have consistent styling regardless of how they were initiated

5. Phase 4: Architecture Hardening

PHASE 4 | ARCHITECTURE HARDENING | WEEK 5

This phase addresses the architectural issues that cause performance problems, runtime crashes, and type safety gaps. While these issues do not directly cause user-visible bugs in the same way as Phase 2 issues, they create a fragile foundation that will make future development increasingly difficult. Addressing them now prevents compounding technical debt.

5.1 Memoize Context Actions and Fix God Object

Priority: High | Effort: 3-4 hours | Risk: Medium

Required Changes:

- In [src/contexts/DataProvider.tsx](#) (line 127), wrap the contextActions object in useMemo, keyed on the baseActions object and all hook return values. This prevents the cascade re-render problem where any state change triggers re-renders across all 8 domain contexts.
- Type the contextActions object properly instead of using "any". Use the ExtendedDataActions type that is already defined but not enforced. This will catch missing handlers at compile time.
- Consider splitting the monolithic AppState into domain-specific slices (e.g., PropertyState, FinanceState, MatterState) with independent update paths. This is a larger refactor that can be deferred, but the memoization fix should be applied immediately.

Acceptance Criteria:

- Changing one property does not cause re-renders in the billing or matter views
- TypeScript catches missing handlers at compile time (no more "actions.X is not a function" at runtime)

5.2 Implement Missing Context Handlers

Priority: High | Effort: 3-4 hours | Risk: Low

Required Changes:

- Implement [handleBulkArchiveTasks](#) and [handleArchiveAllDoneTasks](#) in useTasks hook. These should update task status to "Archived" for the selected tasks.
- Implement [handleUpdateWorkflow](#) in the execution hook. This should update the workflow definition in the workflows array.
- Implement [handleRenamePage](#) and [handleDeleteNotebook](#) in the document hook. These should update the notePages and notebooks arrays respectively.
- For any remaining declared-but-not-implemented handlers (handleUpdateCheckListItem, handleUpdatePageContent), either implement them or remove them from the context type definition to avoid confusion.

Acceptance Criteria:

- TasksView bulk archive does not crash; tasks are moved to archived state
- NotesView page rename does not crash; page names update and persist
- No "actions.X is not a function" errors in the console for any declared handler

5.3 Clean Up Dead State and Duplicate Definitions

Priority: Medium | Effort: 2-3 hours | Risk: Low

Required Changes:

- Remove dead state fields from DataStateContext: isSaving (always false), isOutdated (always false), availableBackups (always []). Either implement these features or remove the declarations to avoid misleading API surface.
- Consolidate the duplicate [EMPTY_APP_STATE](#) definitions. Keep only the one in [src/types.ts](#) and remove the duplicate in [src/utils/mockData.ts](#). Update all imports.
- Remove the hardcoded [appMode: AppMode.Multi](#) in AuthContext. Either implement Solo mode switching or remove the AppMode.Solo enum and associated dead code paths.
- Remove the hardcoded production Convex URL fallback in [src/main.tsx](#) (line 14). The application should fail with a clear error message if VITE_CONVEX_URL is not set, not silently connect to a hardcoded production instance.

Acceptance Criteria:

- No misleading dead state fields in context API
- Single [EMPTY_APP_STATE](#) definition imported consistently
- Application fails clearly if required environment variables are missing

5.4 Incremental Type Safety Improvements

Priority: Medium | Effort: Ongoing | Risk: Low

The codebase contains 100+ "as any" type assertions, concentrated in PropertyDetailView (12+), DataProvider (10+), and demoData.ts (20+). While fixing all of these at once is impractical, a systematic

approach can gradually restore type safety.

Required Changes:

- Start with the highest-risk "as any" usages: those in `PropertyDetailView` that access `property._id`, `property.units`, and `property.name` on loosely-typed objects. Define proper TypeScript interfaces for `Property`, `Unit`, and `Tenant`, and use them consistently.
- Fix the `DataProvider` `contextActions` typing from "any" to the proper `ExtendedDataActions` type (addressed in Task 5.1).
- For `demoData.ts`, define a proper `DemoData` type and cast once at the module boundary, rather than using "as any" on individual fields.
- Replace `Math.random()`-based receipt/invoice number generation with a deterministic sequence. Use a Convex mutation that atomically increments a counter, or use a date-based format like `INV-YYYYMMDD-NNNN`.

Acceptance Criteria:

- No new "as any" assertions are introduced (enforce via lint rule)
- Property domain types are properly defined and used in `PropertyDetailView`
- Receipt and invoice numbers are deterministic and unique

6. Phase 5: Polish and Discovery

PHASE 5 | POLISH AND DISCOVERY | WEEKS 6-7

This phase focuses on user experience polish and discovering what else needs to be built. Once the core workflows are functional and the architecture is sound, the team should conduct a thorough end-to-end review of the property management module to identify gaps that were not visible in the code audit. This phase is intentionally open-ended to accommodate findings from user testing.

6.1 Hydrate Missing Convex Tables

Priority: Medium | Effort: 4-6 hours | Risk: Low

Several Convex tables (`tenancies`, `atrium_inbound_messages`) exist in the schema but are never fetched into `appState`. The Atrium components work around this by querying Convex directly, but this creates inconsistency. This task brings all property-related data into the unified data loading pipeline.

Required Changes:

- Add Convex queries for `tenancies`, `ledger_entries`, `service_charges`, `leads_pipeline`, and `automation_logs` to the `DataProvider`'s data loading pipeline, so that they flow through the context system rather than requiring direct `useConvex()` calls.
- Wire the `tenancies` table into the `PropertyDetailView` and `PropertyForm` for proper tenant management, replacing the current embedded tenant data in property objects.

- Wire `atrium_inbound_messages` into the `AtriumInbox` component.

Acceptance Criteria:

- All Convex tables are loaded through the standard `DataProvider` pipeline
- Atrium components can use context hooks instead of direct Convex queries for data access

6.2 Receipt Template Standardization

Priority: Medium | Effort: 3-4 hours | Risk: Low

Receipts are currently generated in two different ways: `CollectRentModal` uses `reportGenerator.ts` to create a PDF download, while `LedgerManager` opens a new window with raw HTML. Both approaches produce receipts with different layouts and styling. The user has requested that all receipts have the exact same style and standard for all tenants.

Required Changes:

- Create a unified receipt template in `src/services/reportGenerator.ts` that produces consistent output regardless of whether the receipt is for a tenant payment, a management fee disbursement, or a legal invoice payment.
- Replace the `LedgerManager`'s raw HTML receipt generation (`window.open` + `print`) with the unified template.
- Store generated receipt PDFs as documents in Convex (or a file storage service) rather than generating them on-the-fly each time. This allows users to re-download past receipts without regenerating them.

Acceptance Criteria:

- All receipts have identical formatting, typography, and layout
- Past receipts can be re-downloaded without regeneration

6.3 End-to-End User Testing and Gap Discovery

Priority: High | Effort: Ongoing | Risk: N/A

Once Phases 1-4 are complete, conduct a thorough end-to-end user test of the property management module. Walk through the following scenarios and document any issues, confusion points, or missing features that were not caught by the code audit:

- Single-tenant property: Add property, set rent, issue receipt, record payment, log maintenance, view timeline, edit property, view financial reports.
- Multi-unit property: Add property with 5+ units, set per-unit rent and tenants, issue receipts per unit, record payments per unit, log maintenance per unit, view aggregated financial reports, edit individual units, scroll through many units.
- Owner collects model: Set property to "Owner Collects" mode, issue tenant receipts, track management fee disbursements, mark fees as paid, verify financial reporting.

- Manager collects model: Set property to "Manager Collects" mode, collect rent, remit net to owner, generate owner statements.
- Quick Actions flow: Use Quick Actions for all common tasks (issue receipt, rent demand, notice to quit, log maintenance) and verify each action completes in 2-3 clicks.
- Finance integration: Verify property earnings appear in Finance page, can be filtered, and aggregate correctly with legal service earnings.

Document all findings from the end-to-end test and create a follow-up issue list. This list will form the basis for the next iteration of development and may include new features, UX improvements, or edge cases that the code audit could not identify.

7. Testing and Verification Strategy

Each phase should be verified before moving to the next. The following testing strategy ensures that fixes do not introduce regressions and that the property management module works correctly end-to-end.

7.1 Per-Phase Verification Checklist

Phase	Verification Method	Rollback Plan
Phase 1 (Security)	1) Search production bundle for API keys (must find zero). 2) Attempt unauthenticated Convex calls from a fresh client (must be rejected). 3) Test XSS payload in Aloa chat messages (must be sanitized). 4) Check browser console for MFA codes in production build (must not appear).	Revert the Convex deployment if auth breaks existing flows. API key removal can be hot-patched.
Phase 2 (Core Workflows)	1) Add payment in a unit and verify it persists after page refresh. 2) Issue receipt from unit cog menu and verify correct tenant/amount. 3) Log maintenance and verify record creation. 4) Edit a unit and verify changes persist. 5) Verify single-tenant property Quick Actions work.	Keep PropertyTrackingView behind a feature flag so it can be disabled if integration causes issues.
Phase 3 (Financial)	1) Record rent payment and verify it appears in both rent history and ledger. 2) Test "Owner Collects" disbursement flow. 3) Verify Finance page shows property earnings with correct tagging. 4) Compare RevenueEngine totals with individual unit receipts (must match).	Financial data changes require a Convex database backup. Run migration on a staging environment first.

Phase 4 (Architecture)	1) Profile React render counts before and after memoization (must decrease). 2) Test all previously crashing handlers (archive tasks, rename pages, update workflows). 3) Verify TypeScript compilation with no new errors after removing "as any" assertions.	Low risk. Revert individual type changes if they cause compilation issues.
Phase 5 (Polish)	1) Compare receipt PDFs from all generation paths (must be identical). 2) Full end-to-end user test per Section 6.3. 3) Performance audit with large datasets (50+ properties, 200+ units).	N/A - polish phase.

Table 3: Per-Phase Verification Strategy

8. AntiGravity Handoff Notes

This section provides guidance specific to the AntiGravity development team for implementing this plan. It covers coding conventions, architecture constraints, and integration points that should be followed to maintain consistency with the existing codebase.

8.1 Architecture Constraints

- **Context Protocol:** All new state must flow through the existing context system. Do not create standalone state stores or bypass the DataProvider. New data arrays should be added to AppState, new actions should be added to the appropriate hook, and new context values should be exposed through the existing domain contexts.
- **Convex-First Data:** All persistent data must be stored in Convex and loaded through the standard query pipeline. Avoid creating new localStorage-based state or client-only data stores.
- **Hook Pattern:** New hooks must follow the established pattern: accept (appState, actions) as parameters, use try/catch with addToast for error handling, and return named handler functions. Do not introduce hooks with different signatures.
- **Modal Pattern:** New modals must be registered in ModalManager.tsx with a corresponding ModalType enum entry. Modal-specific data must be passed through the modalContext parameter, not through global state or URL parameters.
- **Component Pattern:** New components should use the toolkit components (src/components/toolkit/) for buttons, inputs, and selects. Avoid introducing new UI primitives that duplicate existing ones.

8.2 Key Files Reference

File	Purpose	Modified In
convex/lib/withAuth.ts	Auth middleware (fix immediately)	Phase 1

convex/myFunctions.ts	Core CRUD + admin functions	Phase 1, 4
src/contexts/DataProvider.tsx	Central state + contextActions	Phase 4
src/components/details/PropertyDetailView.tsx	Main property detail page	Phase 2, 3
src/components/details/PropertyTrackingView.tsx	Unit tracking (integrate this)	Phase 2
src/components/modals/ModalManager.tsx	Modal routing + context passing	Phase 2
src/components/modals/CollectRentModal.tsx	Rent collection + receipt generation	Phase 2, 3
src/components/modals/BulkEditPropertyModal.tsx	Bulk property editing	Phase 2
src/components/forms/PropertyForm.tsx	Property/unit creation + editing	Phase 2, 3
src/components/details/ReceiptDetailView.tsx	Receipt display + PDF	Phase 2
src/contexts/FinanceContext.tsx	Finance state (extend for property)	Phase 3
convex/schema.ts	Database schema (add collection model)	Phase 3
src/hooks/useProperties.ts	Property CRUD hooks	Phase 2, 3
src/hooks/useFinance.ts	Finance hooks (extend for property)	Phase 3

Table 4: Key Files Reference for AntiGravity

8.3 Communication Protocol

For each phase, the AntiGravity team should follow this communication protocol to ensure alignment and prevent integration issues:

- Before starting a phase: Confirm understanding of all tasks and acceptance criteria. Flag any ambiguities or concerns before writing code.
- During a phase: Report blockers immediately. If a task requires a larger refactor than estimated, communicate the scope change before proceeding.
- After completing a phase: Provide a summary of what was changed, any deviations from the plan, and confirmation that all acceptance criteria pass. Include screenshots or screen recordings of the fixed workflows.
- After each phase deployment: Conduct a brief regression test across the full property management flow (add property, add units, record payments, issue receipts, log maintenance, view reports) to

catch any unintended side effects.

9. Risk Register

The following risks have been identified for this implementation plan. Each risk includes a mitigation strategy and a contingency plan if the risk materializes.

Risk	Likelihood	Impact	Mitigation
Convex auth changes break existing flows	Medium	High	Deploy auth changes to staging first; test all existing user flows before production
Payment unification causes data loss or duplication	Medium	Critical	Backup Convex database before migration; run migration on staging first; add deduplication logic
PropertyTrackingView integration causes UI regressions	Low	Medium	Feature flag the integration; test with both single and multi-unit properties
Context memoization changes cause stale data reads	Medium	Medium	Add React DevTools profiler before/after comparison; test all CRUD operations
Receipt template changes break existing PDF generation	Low	Low	Keep old generation path as fallback; A/B test with sample receipts
Scope creep from Phase 5 discovery	High	Low	Time-box Phase 5 to 2 weeks; prioritize findings into a separate backlog

Table 5: Risk Register

10. What Comes After: Next Iteration Planning

Once all five phases are complete, the property management module will have a solid foundation. However, the audit identified several areas that are beyond the scope of this plan but should be addressed in subsequent iterations. These represent opportunities for continued improvement and should be prioritized based on user feedback from Phase 5's end-to-end testing.

- **Real Payment Gateway Integration:** Replace the simulated PaymentGatewayModal with an actual payment processor (Paystack or Flutterwave for the Nigerian market). This is a significant feature that requires its own planning cycle.
- **Professional Valuation Module:** Build a proper valuation workflow with history tracking, professional valuation report generation, date tracking, and the ability to compare valuations over

time. This should only appear for properties categorized as "For Sale".

- **Tenant Self-Service Portal:** Allow tenants to view their payment history, download receipts, and submit maintenance requests without going through the property manager. This extends the existing client portal architecture.
- **Automated Rent Reminders:** The AutomationCenter already supports automation rules. Build pre-configured automation templates for rent reminders, lease expiry notices, and maintenance scheduling.
- **Property Analytics Dashboard:** Expand the PropertyReports component into a full analytics dashboard with trend charts, occupancy projections, and ROI calculations.
- **Multi-Currency Support:** While the Naira is currently hardcoded, future expansion may require multi-currency support. The tier constants and NairaSymbol component should be refactored to support configurable currencies.
- **Offline Mode:** The current architecture requires a constant internet connection. Consider adding offline-first capabilities for property inspections and maintenance logging in the field.

The key principle for the next iteration is the same one guiding this plan: keep things neat and simple. Each new feature should integrate cleanly with the existing architecture, follow the established hook protocol, and deliver a clear efficiency improvement without adding confusion. The property management module is close to an architecture that will serve as the foundation going forward. The fixes in this plan are designed to close the remaining gaps and deliver on that promise.